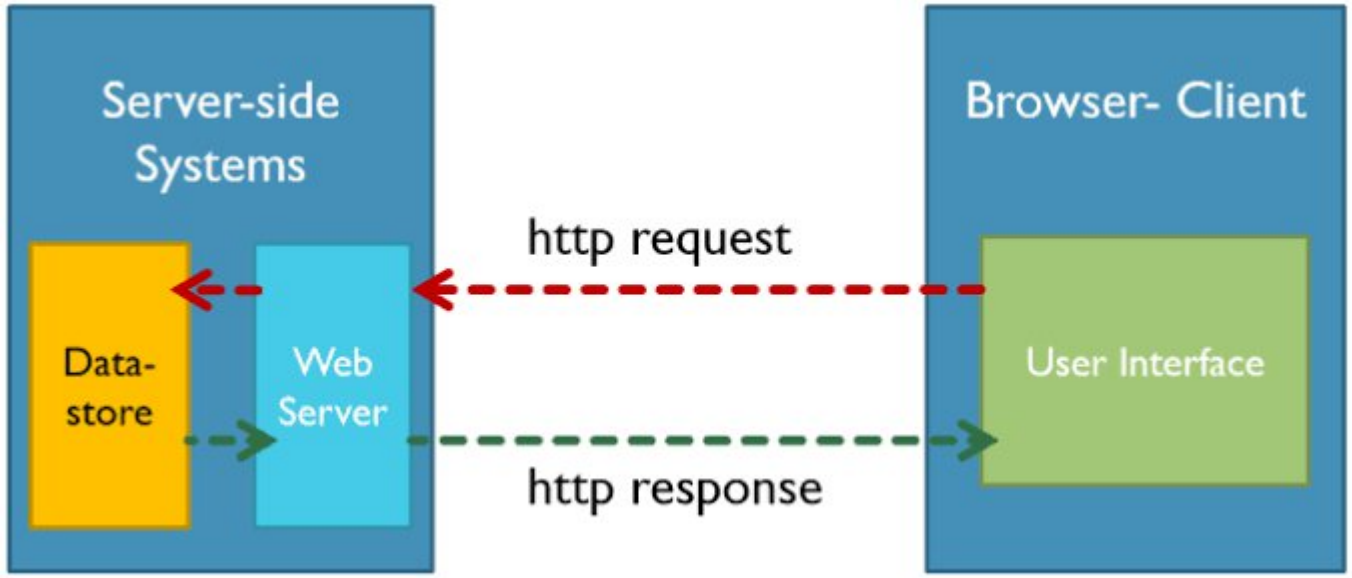
HTTP (HyperText Transfer Protocol)

HTTP (HyperText Transfer Protocol) is the communication protocol used between a web browser (client) and a web server. It defines how requests are sent and how responses are received over the Internet.

- *Client: Sends an HTTP request (e.g., Browser, Mobile App)*
- *Server: Processes the request and sends an HTTP response (e.g., ASP.NET Core application)*



Example

When you type:

```
https://www.google.com
```

The browser sends an HTTP request:

```
GET /
```

Google's server returns an HTTP response containing the HTML page.

Stateless Nature of HTTP

HTTP is **stateless**, meaning the server does **not remember previous requests**.

Example:

Suppose you login to Facebook.

```
Request 1:
POST /login
```

Server verifies your credentials.

Next request:

```
GET /profile
```

The server does **not automatically remember** you logged in.

Therefore websites use:

- Cookies
- Sessions
- Tokens (JWT)

to maintain user login.

HTTP Methods (Verbs)

These define the action to perform.

Method	Purpose	Example
GET	Retrieve data	View student details
POST	Send/Create data	Add new student
PUT	Update entire resource	Update student information
PATCH	Update part of resource	Update student's phone number
DELETE	Delete data	Delete student

HTTP vs HTTPS

HTTP	HTTPS
Not secure	Secure
Data sent in plain text	Data encrypted
Port 80	Port 443
No SSL/TLS	Uses SSL/TLS certificate

Example

```
http://college.com
```

```
https://college.com
```

Always prefer **HTTPS** because it encrypts communication.

HTTP Request Message Format

Whenever a client sends data to the server, it follows a standard format.

```
Request Line

Headers

Blank Line

Body (optional)
```

Diagram

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Chrome
Accept: text/html

(No body for GET)
```

Structure of HTTP Request

1. Request Line

Contains

- HTTP Method
- URL
- HTTP Version

Example

```
GET /students HTTP/1.1
```

Meaning

- GET → retrieve data
- /students → requested resource
- HTTP/1.1 → protocol version

2. Headers

Provide additional information.

Example

```
Host: www.example.com
User-Agent: Chrome
Accept: text/html
```

Accept

Meaning

```
Accept: application/json
```

Client expects JSON.

Host

```
Host: www.example.com
```

Which server to contact.

User-Agent

```
User-Agent: Chrome
```

Which browser is sending request.

3. Blank Line

Separates headers from body.

---

4. Request Body

Present mainly in

- POST
- PUT
- PATCH

Example

```
POST /student HTTP/1.1

Name=Ram
Age=20
```

Complete HTTP Request Example

```
POST /student HTTP/1.1
Host: www.college.com
User-Agent: Chrome
Content-Type: application/json
Content-Length: 32

{
   "Name":"Ram",
   "Age":20
}
```

Explanation

```
POST
```

Create student.

```
Host
```

Server address.

```
Content-Type
```

Sending JSON.

```
Content-Length
```

Size of body.

Body

```
{
"Name":"Ram",
"Age":20
}
```

Actual data sent to server.

HTTP Response Message Format

After receiving a request, the server sends a response.

Format

```
Status Line

Headers

Blank Line

Body
```

Structure of Response

1. Status Line

Contains

- HTTP Version
- Status Code
- Status Message

Example

```
HTTP/1.1 200 OK
```

Meaning

```
HTTP/1.1
```

Protocol version

```
200
```

Status code

```
OK
```

Status message

2. Response Headers

Example

```
Content-Type: text/html
Content-Length: 1500
Date: Wed, 16 July 2026
```

3. Blank Line

Separates headers and body.

4. Response Body

Contains actual content.

Example

```
<html>
<body>
Welcome to KSC
</body>
</html>
```

Complete HTTP Response Example

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 45

<html>
<body>
Hello Student
</body>
</html>
```

Common HTTP Status Codes

Code	Meaning	Example
200 OK	Request successful	Page loaded successfully
201 Created	Resource created	New student added
204 No Content	Success, but no data returned	Delete operation completed
301 Moved Permanently	Resource moved	Website redirects to a new URL
400 Bad Request	Invalid request	Missing required fields in a form
401 Unauthorized	Login required	Accessing a protected page without logging in
403 Forbidden	Access denied	Student tries to access the admin page
404 Not Found	Resource not found	URL does not exist
500 Internal Server Error	Server error	Application crashes due to an exception

Request–Response Example (ASP.NET Core)

Suppose an ASP.NET Core application has a route:

```
GET /students
```

Client Request

```
GET /students HTTP/1.1
Host: localhost:5000
Accept: application/json
```

The browser (or client) sends a GET request to /students, and the ASP.NET Core server processes it and returns a 200 OK response with student data in JSON format.

Server Response

```
HTTP/1.1 200 OK
Content-Type: application/json

[
  {
    "Id":1,
    "Name":"Ram"
  },
  {
    "Id":2,
    "Name":"Sita"
  }
]
```

HTTP Request vs HTTP Response

HTTP Request	HTTP Response
Sent by client (browser/app)	Sent by server
Asks for a resource or performs an action	Returns the requested resource or result
Contains method, URL, headers, and optional body	Contains status line, headers, and optional body
Example: `GET /students`	Example: `HTTP/1.1 200 OK`

Common Web Application Architectures

Feature	Two-Tier	Three-Tier	N-Tier
Layers	2	3	Many
Complexity	Low	Medium	High
Performance	Good	Good	Good
Security	Low	High	Very High
Scalability	Poor	Good	Excellent
Best For	Small apps	Medium applications	Large enterprise systems

MVC Pattern (Model-View-Controller)

MVC (Model-View-Controller) is a software design pattern used to separate an application into three independent components. It improves code organization, maintainability, reusability, and testing.

1. Model

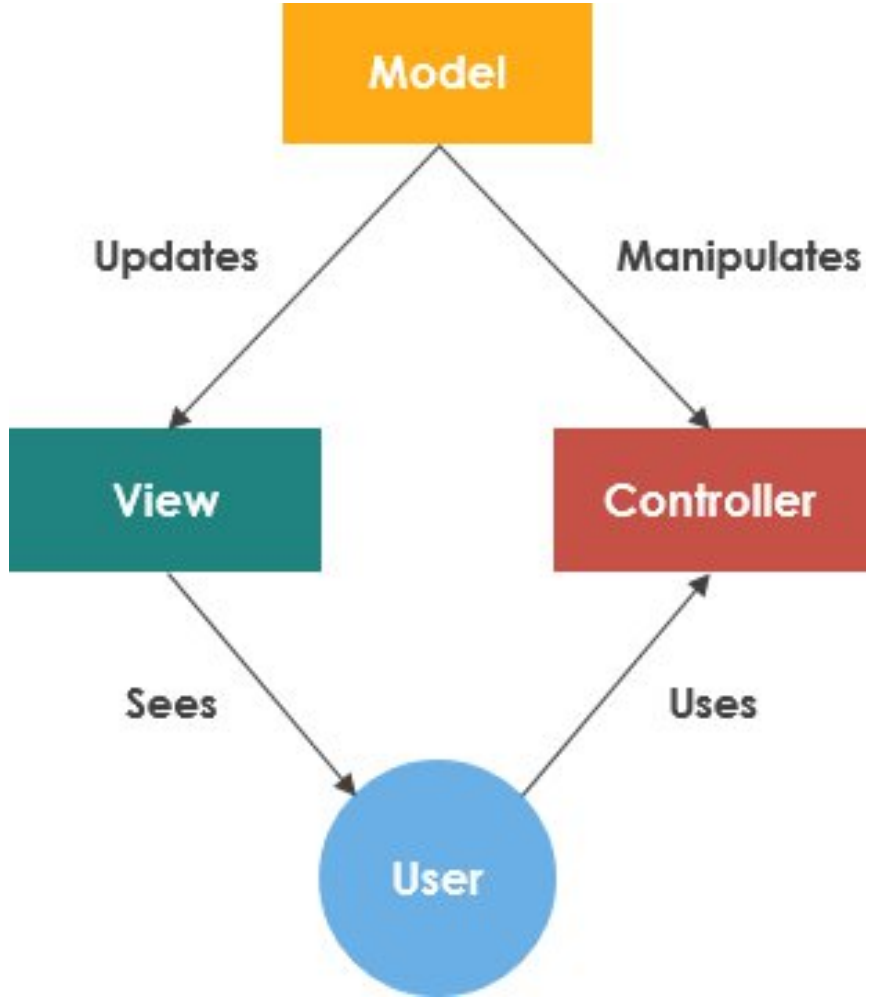
The **Model** represents the application's **data and business logic**.

Responsibilities

- Stores data
- Retrieves data from database
- Performs business rules
- Updates data

```
public class Student
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

The `Student` class is the **Model**.



2. View

The **View** is the **User Interface (UI)**.

Responsibilities

- Displays information
- Receives user input
- Contains HTML, CSS, Razor pages

Example

HTML

```html
<h2>Student List</h2>

<p>Ram</p>
<p>Sita</p>
```

This page only displays data.

3. Controller

The **Controller** acts as the **middleman** between the Model and the View.

Responsibilities

- Receives user request
- Calls Model
- Sends data to View

Example

C#

```csharp
public class StudentController : Controller
{
    public IActionResult Index()
    {
        return View();
    }
}
```

MVC Working Process

Suppose the user visits:

```
https://college.com/students
```

Step 1

Browser sends request

```
GET /students
```

↓

Step 2

Controller receives request.

```
StudentController
```

↓

Step 3

Controller asks Model for data.

```
Student Model
```

↓

Step 4

Model retrieves data from database.

```
Ram
Sita
Hari
```

↓

Step 5

Controller sends data to View.

↓

Step 6

View generates HTML.

↓

Step 7

Browser displays webpage.

Note: Create Project with ASP.NET Core Web App (Model-View-Controller) in Visual Studio; Do not choose "ASP.NET Core Web App (Razor Pages)". Choose the Model-View-Controller (MVC) template.

Suppose a student list application.

Model

C#

```
public class Student
{
    public string Name { get; set; }
}
```

View (Index.cshtml)

HTML

```
@model List<Student>

<h2>Students</h2>

@foreach(var s in Model)
{
    <p>@s.Name</p>
}
```

Controller

C#

```
public class StudentController : Controller
{
    public IActionResult Index()
    {
        List<Student> students = new List<Student>()
        {
            new Student(){Name="Ram"},
            new Student(){Name="Sita"}
        };

        return View(students);
    }
}
```

Output

```
Students

Ram
Sita
```

Advantages of MVC

- Separates application into **Model, View, and Controller**.
- Easier to maintain and modify.
- Supports code reuse.
- Easy unit testing.
- Multiple developers can work independently on different components.
- Well suited for large web applications.

Disadvantages of MVC

- More files and folders than simple web applications.
- Slightly difficult for beginners.
- May be unnecessary for very small projects.

MVC vs Traditional Web Application

Traditional Web Application	MVC Pattern
UI and business logic mixed together	Clear separation of concerns
Difficult to maintain	Easy to maintain
Less reusable code	Highly reusable
Difficult to test	Easy to unit test
Suitable for small applications	Suitable for medium and large applications

- *Describe the procedure of deploying .NET core application.*
- *Differentiate between .NET, .NET Core, and Mono frameworks. Explain the compilation and execution process of .NET applications covering CLI, MSIL, and CLR.*
- *Describe the ASP.NET Core architecture overview. How does it differ from traditional ASP.NET framework architecture?*
- *Describe the importance of MVC pattern in designing web applications.*
- *Differentiate between .NET and ASP.NET frameworks. How do you test the .NET core applications?*
- *Define HTTP. Explain the MVC pattern.*
- *Explain about request and response message format with example.*
- *Explain the process to deploy the core application.*